RECONOCIMIENTO DE OBJETOS CON YOLO

GRUPO 3: Geovanna Calahorrano

Ivan Vega Salazar

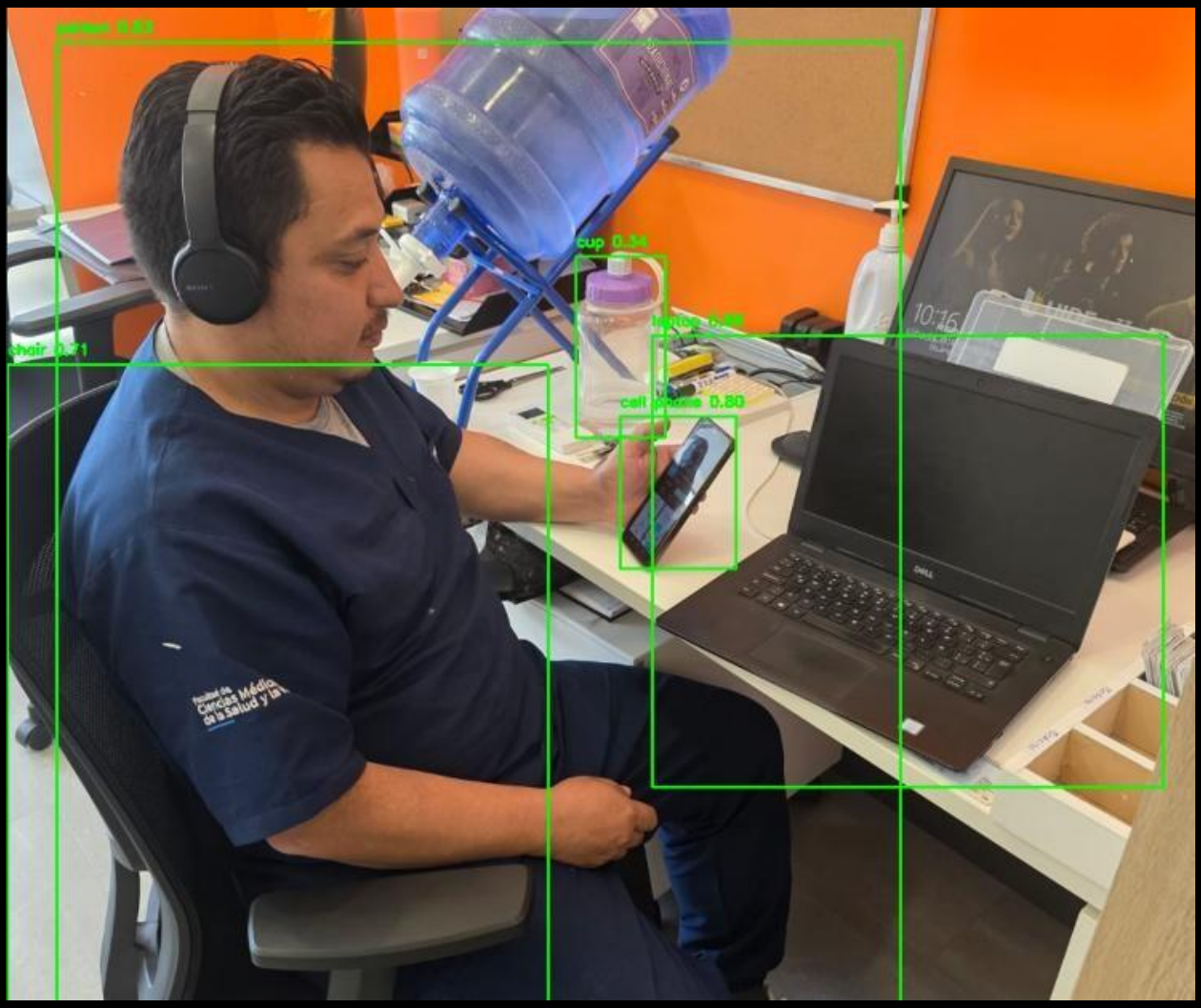
Carlos Yanez

```
!pip install ultralytics
from ultralytics import YOLO
import cv2
from google.colab.patches import cv2_imshow # Import cv2_imshow for Colab compa

# Cargar modelo
modelo = YOLO("yolov8n.pt")

# Clases permitidas
clases_permitidas = ["person", "cell phone", "laptop", "cup", "chair", "remote"

# Activar cámara (0 = webcam)
camara = cv2.VideoCapture(0)

# Only attempt to run webcam if it's available
# Note: Webcam access in Colab requires user permission and might not be stable
if camara.isOpened():
    print("Webcam is open. Press 'q' to quit.")
    while True:
        ret, cuadro = camara.read()
        if not ret:
            print("Failed to grab frame from camera.")
            break

        resultados = modelo(cuadro)

        for resultado in resultados:
            for caja in resultado.boxes:
                id_clase = int(caja.cls[0])
                nombre_clase = modelo.names[id_clase]
                confianza = float(caja.conf[0])

                if nombre_clase in clases_permitidas:
                    x1, y1, x2, y2 = map(int, caja.xyxy[0])

                    cv2.rectangle(cuadro, (x1, y1), (x2, y2), (0, 255, 0), 2)
                    texto = f"{nombre_clase} {confianza:.2f}"
```

```
                    cv2.putText(cuadro, texto, (x1, y1 - 10),
                                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

        cv2_imshow(cuadro) # Use cv2_imshow instead of cv2.imshow

        # Use a short delay for webcam stream
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    camara.release()
    cv2.destroyAllWindows()
else:
    print("Could not open webcam. Skipping webcam detection.")


# --- Image Detection --- #

# Cargar modelo (already loaded above, no need to reload)
# modelo = YOLO("yolov8n.pt")

# Clases permitidas (already defined above, no need to redefine)
# clases_permitidas = ["person", "cell phone", "laptop"]

# Ruta de imagen
ruta_imagen = "/content/Imagen tarea2.jpeg"

imagen = cv2.imread(ruta_imagen)

if imagen is None:
    print(f"Error: Image '{ruta_imagen}' not found. Please upload the image file or provide a correct path.")
else:
    resultados = modelo(imagen)

    for resultado in resultados:
        for caja in resultado.boxes:
            id_clase = int(caja.cls[0])
            nombre_clase = modelo.names[id_clase]
            confianza = float(caja.conf[0])
```

```
            if nombre_clase in clases_permitidas:
                x1, y1, x2, y2 = map(int, caja.xyxy[0])

                cv2.rectangle(imagen, (x1, y1), (x2, y2), (0, 255, 0), 2)
                texto = f"{nombre_clase} {confianza:.2f}"

                cv2.putText(imagen, texto, (x1, y1 - 10),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

    cv2_imshow(imagen) # Use cv2_imshow instead of cv2.imshow
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

```
Requirement already satisfied: ultralytics-thop>=2.0.18 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.18)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (26.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (2.9.0.post0)
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.12/dist-packages (from polars>=0.20.0->ultralytics) (1.35.2)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (3.4.6)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (2026.2.25)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.25.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (2025.3.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib>=3.3.0->ultralytics) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.8.0->ultralytics) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.8.0->ultralytics) (3.0.3)
Could not open webcam. Skipping webcam detection.

0: 640x480 1 person, 1 cup, 1 chair, 1 laptop, 1 keyboard, 1 cell phone, 372.7ms
```


cup 0.34
cell phone 0.80
laptop
DELL

```python
!pip install ultralytics
from ultralytics import YOLO
import cv2

# El modelo y las clases permitidas ya están cargadas/definidas en celdas anteriores
# modelo = YOLO("yolov8n.pt")
# clases_permitidas = ["person", "cell phone", "laptop", "cup", "chair", "remote"]

# Rutas de los videos
ruta_video_entrada = "/content/video tarea2.mp4" # <--- ACTUALIZA CON TU VIDEO DE ENTRADA
ruta_video_salida = "/content/video_con_detecciones.mp4" # <--- RUTA PARA EL VIDEO DE SALIDA

# Abrir el archivo de video de entrada
cap = cv2.VideoCapture(ruta_video_entrada)

# Verificar si el video se abrió correctamente
if not cap.isOpened():
    print(f"Error: No se pudo abrir el archivo de video de entrada en '{ruta_video_entrada}'.")
else:
    # Obtener las propiedades del video de entrada
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    fps = int(cap.get(cv2.CAP_PROP_FPS))
    codec = cv2.VideoWriter_fourcc(*'mp4v') # Codec para .mp4, en Colab a menudo se usa 'mp4v'

    # Crear el objeto VideoWriter para guardar el video de salida
    out = cv2.VideoWriter(ruta_video_salida, codec, fps, (width, height))

    print(f"Procesando video: {ruta_video_entrada}. Guardando en: {ruta_video_salida}")
    while True:
        ret, frame = cap.read()
        if not ret:
            print("Fin del video o error al leer el cuadro.")
            break

        # Realizar detección en el cuadro actual
        resultados = modelo(frame)
```

```python
        # Dibujar resultados en el cuadro
        for resultado in resultados:
            for caja in resultado.boxes:
                id_clase = int(caja.cls[0])
                nombre_clase = modelo.names[id_clase]
                confianza = float(caja.conf[0])

                if nombre_clase in clases_permitidas:
                    x1, y1, x2, y2 = map(int, caja.xyxy[0])

                    cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
                    texto = f"{nombre_clase} {confianza:.2f}"

                    cv2.putText(frame, texto, (x1, y1 - 10),
                                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

        # Escribir el cuadro procesado en el archivo de salida
        out.write(frame)

    # Liberar los objetos VideoCapture y VideoWriter
    cap.release()
    out.release()
    cv2.destroyAllWindows()
    print("Procesamiento de video completado y guardado.")
```

```
Requirement already satisfied: ultralytics in /usr/local/lib/python3.12/dist-packages (8.4.24)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.2)
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (3.10.0)
Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (4.13.0.92)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (11.3.0)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.32.4)
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.16.3)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.10.0+cpu)
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (0.25.0+cpu)
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (5.9.5)
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.35.2)
Requirement already satisfied: ultralytics-thop>=2.0.18 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.18)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.3.3)
```

Video Object Detection

This section demonstrates how to perform real-time object detection on a video file. The code will:

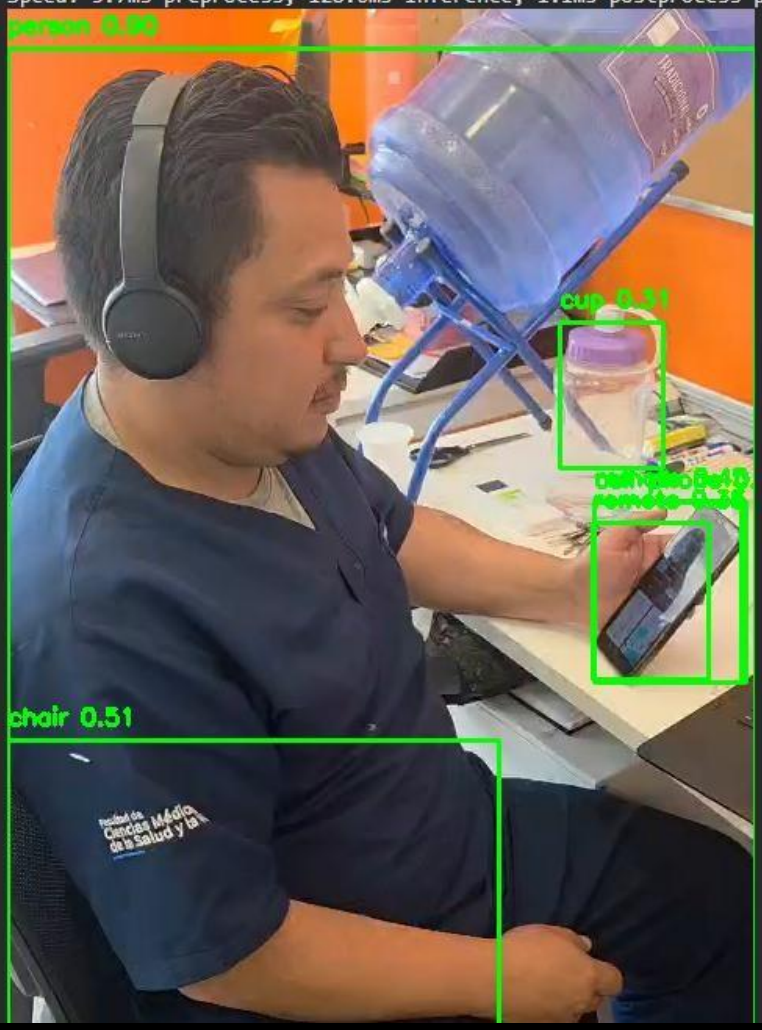
1. Load the YOLOv8n model.
2. Open a video file.
3. Process each frame of the video.
4. Detect objects belonging to the `clases_permitidas` list.
5. Draw bounding boxes and labels on the detected objects.
6. Display the processed video frame by frame.

Remember to replace `ruta_video` with the actual path to your video file.

```
from ultralytics import YOLO
import cv2
from google.colab.patches import cv2_imshow # Import cv2_imshow for Colab compatibility

# Cargar modelo (re-using the already loaded model is more efficient)
# modelo = YOLO("yolov8n.pt")

# Clases permitidas (re-using already defined classes)
# clases_permitidas = ["person", "cell phone", "laptop"]

# Ruta del video
ruta_video = "/content/video tarea2.mp4" # <--- UPDATE THIS WITH YOUR VIDEO PATH

# Abrir el archivo de video
video = cv2.VideoCapture(ruta_video)

# Verificar si el video se abrió correctamente
if not video.isOpened():
    print(f"Error: No se pudo abrir el archivo de video en '{ruta_video}'. Asegúrate de que el archivo existe y la ruta es correcta.")
else:
    print(f"Procesando video: {ruta_video}. Presiona 'q' para salir.")
    while True:
        ret, cuadro = video.read()
        if not ret:
            print("Fin del video o error al leer el cuadro.")
            break

        # Realizar detección en el cuadro actual
        resultados = modelo(cuadro)

        # Dibujar resultados en el cuadro
        for resultado in resultados:
            for caja in resultado.boxes:
                id_clase = int(caja.cls[0])
                nombre_clase = modelo.names[id_clase]
                confianza = float(caja.conf[0])
```

```
            if nombre_clase in clases_permitidas:
                x1, y1, x2, y2 = map(int, caja.xyxy[0])

                cv2.rectangle(cuadro, (x1, y1), (x2, y2), (0, 255, 0), 2)
                texto = f"{nombre_clase} {confianza:.2f}"

                cv2.putText(cuadro, texto, (x1, y1 - 10),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

        # Mostrar el cuadro con las detecciones
        cv2_imshow(cuadro)

        # Salir si se presiona 'q'
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    # Liberar el objeto VideoCapture y destruir todas las ventanas
    video.release()
```

```
Procesando video: /content/video tarea2.mp4. Presiona 'q' para salir.

0: 640x384 1 person, 1 cup, 1 chair, 2 remotes, 1 cell phone, 128.6ms
Speed: 5.7ms preprocess, 128.6ms inference, 1.1ms postprocess per image at shape (1, 3, 640, 384)
```



Uso en situaciones reales:

El uso inadecuado de teléfonos inteligentes durante la jornada laboral puede generar interrupciones frecuentes, disminución del rendimiento operativo y, en algunos casos, incumplimiento de protocolos establecidos por la organización. Estas conductas, aunque comunes, suelen ser difíciles de monitorear de manera continua sin recurrir a mecanismos automatizados que garanticen objetividad y precisión.

En este contexto, la implementación de sistemas de visión por computadora basados en algoritmos de detección en tiempo real, con **YOLO (You Only Look Once)**, constituye una herramienta altamente eficiente para la supervisión automatizada. Este modelo permite identificar de forma simultánea objetos y personas dentro de un entorno determinado, diferenciando entre actividades permitidas y comportamientos que podrían afectar la productividad o vulnerar políticas internas.

El uso de en este caso nos permite el **Procesamiento en tiempo real**, lo que permite generar alertas o registros instantáneos.

- **Alta precisión en la detección**, incluso en condiciones de iluminación variable o con múltiples personas en escena.
- **Capacidad de clasificación de objetos específicos**, como dispositivos móviles, herramientas de trabajo o elementos no autorizados.
- **Generación de evidencia visual**, útil para auditorías internas, análisis de incidentes o toma de decisiones administrativas.

Gracias a estas capacidades, la organización puede obtener información estructurada y respaldada por evidencia objetiva, reduciendo la subjetividad en los procesos de supervisión y fortaleciendo los mecanismos de control. En consecuencia, la integración de YOLO no solo optimiza la gestión del entorno laboral, sino que también contribuye a mejorar la eficiencia operativa y el cumplimiento de las políticas corporativas.